# Reporte: 2p ejercicio 20(poo)

En este programa implementamos el algoritmo Merge Sort utilizando la técnica de Ordenamiento Indirecto bajo el paradigma de Programación Orientada a Objetos. En lugar de mover o intercambiar los datos originales en memoria, ordenamos un arreglo secundario de índices. De esta manera, el vector con la información original permanece intacto mientras que el vector de índices determina la secuencia ordenada de acceso.

### Código Completo:

```
#include <iostream>
#include <vector>
#include <string>
#include <limits>

// Nuevo tipo de dato como estructura genérica para POO
struct Registro {
    int id;
    double datoNumerico;
};

// ------------------------------------------------------------
// CLASE GESTOR DE ORDENAMIENTO INDIRECTO (POO)
// ------------------------------------------------------------
class OrdenamientoIndirectoPOO {
private:
    // Merge Indirecto para Enteros
    void mergeEnteros(const std::vector<int>& datos, std::vector<int>& indices, int izq,
int medio, int der) {
        int n1 = medio - izq + 1;
        int n2 = der - medio;
        std::vector<int> L(n1), R(n2);

        for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
        for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];

        int i = 0, j = 0, k = izq;
        while (i < n1 && j < n2) {
            if (datos[L[i]] <= datos[R[j]]) {
                indices[k++] = L[i++];
            } else {
                indices[k++] = R[j++];
            }
        }
        while (i < n1) indices[k++] = L[i++];
        while (j < n2) indices[k++] = R[j++];
    }

    void mergeSortEnterosHelper(const std::vector<int>& datos, std::vector<int>& indices,
int izq, int der) {
        if (izq < der) {
            int medio = izq + (der - izq) / 2;
            mergeSortEnterosHelper(datos, indices, izq, medio);
            mergeSortEnterosHelper(datos, indices, medio + 1, der);
            mergeEnteros(datos, indices, izq, medio, der);
```

```
        }
    }

    // Merge Indirecto para Caracteres
    void mergeCaracteres(const std::vector<char>& datos, std::vector<int>& indices, int
izq, int medio, int der) {
        int n1 = medio - izq + 1;
        int n2 = der - medio;
        std::vector<int> L(n1), R(n2);

        for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
        for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];

        int i = 0, j = 0, k = izq;
        while (i < n1 && j < n2) {
            if (datos[L[i]] <= datos[R[j]]) {
                indices[k++] = L[i++];
            } else {
                indices[k++] = R[j++];
            }
        }
        while (i < n1) indices[k++] = L[i++];
        while (j < n2) indices[k++] = R[j++];
    }

    void mergeSortCaracteresHelper(const std::vector<char>& datos, std::vector<int>&
indices, int izq, int der) {
        if (izq < der) {
            int medio = izq + (der - izq) / 2;
            mergeSortCaracteresHelper(datos, indices, izq, medio);
            mergeSortCaracteresHelper(datos, indices, medio + 1, der);
            mergeCaracteres(datos, indices, izq, medio, der);
        }
    }

    // Merge Indirecto para Nuevo Tipo de Dato (Registro)
    void mergeRegistros(const std::vector<Registro>& datos, std::vector<int>& indices,
int izq, int medio, int der) {
        int n1 = medio - izq + 1;
        int n2 = der - medio;
        std::vector<int> L(n1), R(n2);

        for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
        for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];

        int i = 0, j = 0, k = izq;
        while (i < n1 && j < n2) {
            if (datos[L[i]].id <= datos[R[j]].id) {
                indices[k++] = L[i++];
            } else {
                indices[k++] = R[j++];
            }
        }
        while (i < n1) indices[k++] = L[i++];
        while (j < n2) indices[k++] = R[j++];
    }

    void mergeSortRegistrosHelper(const std::vector<Registro>& datos, std::vector<int>&
```

```
indices, int izq, int der) {
        if (izq < der) {
            int medio = izq + (der - izq) / 2;
            mergeSortRegistrosHelper(datos, indices, izq, medio);
            mergeSortRegistrosHelper(datos, indices, medio + 1, der);
            mergeRegistros(datos, indices, izq, medio, der);
        }
    }

public:
    // Método público para gestionar Enteros
    void ejecutarEnteros() {
        int n;
        std::cout << "Cantidad de enteros: ";
        std::cin >> n;
        if (n <= 0) return;

        std::vector<int> datos(n);
        std::vector<int> indices(n);

        for (int i = 0; i < n; i++) {
            std::cout << "Entero [" << i << "]: ";
            std::cin >> datos[i];
            indices[i] = i;
        }

        mergeSortEnterosHelper(datos, indices, 0, n - 1);

        std::cout << "\n--- DATOS ORIGINALES ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << datos[i] << " ";
        std::cout << "]\n";

        std::cout << "--- ÍNDICES ORDENADOS ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << indices[i] << " ";
        std::cout << "]\n";

        std::cout << "--- ACCESO INDIRECTO ORDENADO ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << datos[indices[i]] << " ";
        std::cout << "]\n";
    }

    // Método público para gestionar Caracteres
    void ejecutarCaracteres() {
        int n;
        std::cout << "Cantidad de caracteres: ";
        std::cin >> n;
        if (n <= 0) return;

        std::vector<char> datos(n);
        std::vector<int> indices(n);

        for (int i = 0; i < n; i++) {
            std::cout << "Caracter [" << i << "]: ";
            std::cin >> datos[i];
            indices[i] = i;
        }

        mergeSortCaracteresHelper(datos, indices, 0, n - 1);
```

```
        std::cout << "\n--- DATOS ORIGINALES ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << datos[i] << " ";
        std::cout << "]\n";

        std::cout << "--- ÍNDICES ORDENADOS ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << indices[i] << " ";
        std::cout << "]\n";

        std::cout << "--- ACCESO INDIRECTO ORDENADO ---\n[ ";
        for (int i = 0; i < n; i++) std::cout << datos[indices[i]] << " ";
        std::cout << "]\n";
    }

    // Método público para gestionar Registros (Nuevo Tipo de Dato)
    void ejecutarRegistros() {
        int n;
        std::cout << "Cantidad de registros: ";
        std::cin >> n;
        if (n <= 0) return;

        std::vector<Registro> datos(n);
        std::vector<int> indices(n);

        for (int i = 0; i < n; i++) {
            std::cout << "\nRegistro " << i << ":\n";
            std::cout << "  ID (criterio de orden): ";
            std::cin >> datos[i].id;
            std::cout << "  Dato numérico: ";
            std::cin >> datos[i].datoNumerico;
            indices[i] = i;
        }

        mergeSortRegistrosHelper(datos, indices, 0, n - 1);

        std::cout << "\n--- ACCESO INDIRECTO ORDENADO POR ID ---\n";
        for (int i = 0; i < n; i++) {
            int idx = indices[i];
            std::cout << "Posición " << i << " -> Índice original " << idx
                      << " | ID: " << datos[idx].id << " | Dato numérico: " <<
datos[idx].datoNumerico << "\n";
        }
    }
};

int main() {
    OrdenamientoIndirectoPOO ordenador;
    int opcion = 0;

    do {
        std::cout << "\n==================================================\n";
        std::cout << "   MERGE SORT INDIRECTO (POO) - MENÚ INTERACTIVO   \n";
        std::cout << "==================================================\n";
        std::cout << "1. Ordenamiento indirecto de Enteros\n";
        std::cout << "2. Ordenamiento indirecto de Caracteres\n";
        std::cout << "3. Ordenamiento indirecto de Nuevo Tipo de Dato\n";
        std::cout << "4. Salir\n";
        std::cout << "Seleccione una opción: ";
```

```
        if (!(std::cin >> opcion)) break;

        switch (opcion) {
            case 1:
                ordenador.ejecutarEnteros();
                break;
            case 2:
                ordenador.ejecutarCaracteres();
                break;
            case 3:
                ordenador.ejecutarRegistros();
                break;
            case 4:
                std::cout << "Saliendo del programa...\n";
                break;
            default:
                std::cout << "Opción no válida.\n";
        }

    } while (opcion != 4);

    return 0;
}
```

## Explicación del Código:

**1. Estructura de Dato Personalizado (struct Registro)**
Define una estructura genérica con los campos id y datoNumerico. Permite demostrar que el ordenamiento indirecto funciona eficientemente para tipos de datos compuestos u objetos sin necesidad de mover estructuras pesadas en memoria.

**2. Concepto de Ordenamiento Indirecto en Merge Sort (private:)**
En los métodos de mezcla (mergeEnteros, mergeCaracteres, mergeRegistros), la comparación utiliza los valores originales a través de los índices (datos[L[i]] <= datos[R[j]]), pero los intercambios y la reescritura ocurren únicamente sobre el vector de índices (indices[k++] = L[i++]). Las funciones auxiliares helper dividen la secuencia de índices de forma recursiva.

**3. Métodos Públicos de Interfaz (ejecutarEnteros, ejecutarCaracteres, ejecutarRegistros)**
Se encargan de capturar los datos desde la consola e inicializar el vector de índices de 0 a n-1. Tras ejecutar el ordenamiento, imprimen el vector original (sin modificaciones), el vector de índices ordenados y el resultado final accediendo de forma indirecta mediante datos[indices[i]].

**4. Clase OrdenamientoIndirectoPOO y Función Principal (main)**
Encapsula toda la lógica del algoritmo dentro de una clase. La función main crea un objeto ordenador y maneja la navegación por el menú interactivo mediante un ciclo do-while y una estructura de control switch-case.